

CipherSchools — Low-Level Design (LLD) Practice Platform

 **Live Demo Platform:** <https://lowleveldesign.vercel.app>

 **Live Backend API:** <https://low-level-design.onrender.com/api> (Health Check: `/api/health`)

An interactive, deliberate-practice platform for mastering Low-Level Design (LLD) and Object-Oriented Design (OOD). Built for the **CipherSchools Hiring Assignment (September 2026)**.

The Learner Problem Solved

Unlike algorithmic challenges where automated test cases yield clear Pass/Fail signals, Low-Level Design is notoriously difficult to self-evaluate. Candidates sketch classes and design patterns but have no objective way of knowing whether they created god-objects, coupled interfaces prematurely, or neglected critical concurrency edge cases. Generic chatbots offer superficial, uncalibrated praise without structured rubrics or evidence grounding.

This platform bridges that gap by providing:

- Curated Problem Library:** 4 real-world systems with explicit requirements and constraints (Parking Lot, Elevator System, Vending Machine, Food Delivery App).
- Standardized 100-Point Rubric:** Evaluates solutions across 7 architectural dimensions dynamically served from the database.
- Evidence-Grounded Feedback:** Every score is backed by verbatim quotations from the student's submission, concrete concerns, actionable suggestions, and confidence ratings.
- Save-Before-Evaluate Reliability:** Submissions are committed to PostgreSQL *before* calling the evaluator, guaranteeing student work is never lost on network or model failures.
- Measurable Improvement Loop:** Attempts are immutable records. When learners retry, previous attempts are preserved, and score trajectory badges (`+15 pts` , `-5 pts` , `Baseline`) highlight progress.

Architectural Highlights

- Relational Integrity via PostgreSQL & Prisma:** Strictly enforces 1:1 Attempt-to-Submission and 1:1 Submission-to-Evaluation relationships via database-level `UNIQUE` constraints and foreign keys.
- Save-Before-Evaluate Guarantee:** Submissions are durably stored in PostgreSQL before the evaluation begins. If the evaluator fails, the submission is never lost and evaluation can be retried in-place.
- Vendor-Agnostic Evaluator Abstraction:** Implements an `Evaluator` domain contract supporting `GeminiProvider` (`gemini-3.6-flash`), `OpenAIProvider` (`gpt-4o-mini`), and `MockLLMProvider` (for deterministic offline testing).
- Zod Output Validation & Grounding Engine:** Rejects hallucinated criteria, ungrounded placeholder evidence, duplicate keys, and arithmetic errors. Deterministically computes total scores on the server.
- Concurrent Safe Execution:** Server-level concurrency control prevents race conditions on duplicate evaluation requests.

Tech Stack

- Frontend:** React 19, TypeScript, React Router v7, Vite 8, Vitest, Testing Library, Vanilla CSS (Dark Workbench design system).
- Backend:** Node.js, Express 4, TypeScript, Prisma ORM, Zod, Vitest.
- Database:** PostgreSQL (Prisma Client).
- AI / Evaluation:** Google Gemini (configurable via `GEMINI_MODEL` , default: `gemini-3.6-flash`) via structured `Evaluator` abstraction; OpenAI (`gpt-4o-mini`); offline `MockLLMProvider` for deterministic testing.
- Testing:** 70 automated tests (61 backend unit/domain/api tests + 9 frontend component/flow tests).

Directory & Command Map

Every command in this repository must be executed from its specific directory:

Directory	Purpose	Allowed Commands
backend/	Express API, Prisma ORM, Evaluator services, Domain logic	<code>npm install</code> , <code>npx prisma generate</code> , <code>npx prisma migrate dev</code> , <code>npm run db:seed</code> , <code>npm test</code> , <code>npx tsc --noEmit</code> , <code>npm run dev</code>
frontend/	React single-page application, Dark Workbench UI, Vitest	<code>npm install</code> , <code>npm test</code> , <code>npm run build</code> , <code>npm run lint</code> , <code>npm run dev</code>
Repository Root	Documentation, architectural notes, research briefs	Git operations, viewing documentation

Local Setup & Clean-Install Guide

Prerequisites

- **Node.js:** v20+ or v22+
- **PostgreSQL:** v14+ running locally (default port: `5432`)

1. Database Setup

Create the database:

```
createdb lld_practice
# or using psql:
# psql -U postgres -c "CREATE DATABASE lld_practice;"
```

2. Backend Setup

From the repository root:

```
cd backend
npm install
cp .env.example .env
```

Configure `backend/.env` with your PostgreSQL connection:

```
PORT=3001
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/lld_practice?schema=public"

# For offline testing or local evaluation without API keys:
LLM_PROVIDER=mock

# For live evaluations with Google Gemini:
# LLM_PROVIDER=gemini
# GEMINI_API_KEY=your_gemini_api_key_here
# GEMINI_MODEL=gemini-3.6-flash

# For live evaluations with OpenAI:
# LLM_PROVIDER=openai
# OPENAI_API_KEY=your_openai_api_key_here
# OPENAI_MODEL=gpt-4o-mini
```

Run database migrations, generate Prisma Client, and seed the 4 core problems:

```
npx prisma migrate dev
npm run db:seed
```

Run backend automated tests (100% offline with MockLLMProvider):

```
npm test
```

Start the backend API server:

```
npm run dev
```

The backend starts on `http://localhost:3001` . Verify with `curl http://localhost:3001/api/health` .

3. Frontend Setup

In a separate terminal window:

```
cd frontend
npm install
npm test
npm run dev
```

Open `http://localhost:5173` in your browser.

Reviewer Demo Walkthrough

Follow this step-by-step path to experience the complete practice loop:

- 1. **Problem Library:** Navigate to `http://localhost:5173` . Browse the 4 curated LLD problems (Parking Lot, Elevator System, Vending Machine, Food Delivery). Note difficulty tags and requirement counts.
- 2. **Problem Detail:** Click on **"Practice →"** on *Design a Parking Lot*. Read the explicit requirements (spots, vehicle types, ticketing, concurrency) and system constraints.
- 3. **Start Attempt:** Click **"Start Practice Attempt →"**. Notice the structured starter template with sections for Requirements, Domain Entities, Design Patterns, Sequences, Extensibility, and Edge Cases.
- 4. **Dynamic Rubric:** Observe the right-hand panel displaying the 7 evaluation dimensions dynamically loaded from the backend (`GET /api/rubric`) totaling 100 points.
- 5. **Submit Solution:** Write or refine your solution and click **"Submit Design for Evaluation →"**.
- 6. **Save-Before-Evaluate & Evaluation:** The submission is committed atomically before evaluation begins. The evaluator assesses the solution and renders:
 - Overall score out of 100 (deterministically aggregated).
 - 7 criterion cards with verbatim quotes of evidence, identified architectural concerns, actionable recommendations, and evaluator confidence.
- 7. **Iterative Refinement (Try Again):**
 - Click **"Try Again (Start New Attempt) →"**.
 - Attempt #1 is permanently archived.
 - Enter an improved solution (e.g. adding a `PaymentStrategy` or `SpotAllocationStrategy`).
 - Submit again: View the updated Attempt History showing Attempt #1 vs. Attempt #2 with comparative score badges (`+15 pts`).

Environment Variables

Variable	Directory	Required	Description	Default / Example
<code>PORT</code>	<code>backend/</code>	No	Backend HTTP port	<code>3001</code>
<code>DATABASE_URL</code>	<code>backend/</code>	Yes	PostgreSQL connection string	<code>postgresql://user:pass@localhost:5432/lld_practice</code>
<code>LLM_PROVIDER</code>	<code>backend/</code>	No	Evaluation provider (<code>mock</code> , <code>gemini</code> , <code>openai</code>)	<code>mock</code> (tests) / <code>gemini</code> (live)
<code>GEMINI_API_KEY</code>	<code>backend/</code>	Conditional	Google Gemini API key (required if <code>LLM_PROVIDER=gemini</code>)	<code>AIzaSy...</code>
<code>GEMINI_MODEL</code>	<code>backend/</code>	No	Gemini model identifier	<code>gemini-3.6-flash</code>
<code>OPENAI_API_KEY</code>	<code>backend/</code>	Conditional	OpenAI API key (required if <code>LLM_PROVIDER=openai</code>)	<code>sk-...</code>

Variable	Directory	Required	Description	Default / Example
OPENAI_MODEL	backend/	No	OpenAI model identifier	gpt-4o-mini
VITE_API_BASE_URL	frontend/	No	Optional API base URL for production proxy	/api

Testing & Quality Verification

All automated tests run completely offline without external network or LLM API calls using `MockLLMProvider`.

Backend Test Suite (61 tests):

```
cd backend && npm test
```

```
✓ tests/llm.evaluator.test.ts (18 tests) – Evidence grounding, prompt builder, retry, sanitization
✓ tests/api.test.ts (18 tests) – REST endpoints, retry, concurrency, rubric consistency
✓ tests/domain.test.ts (11 tests) – Domain constraints, 1:1 invariants, state machine
✓ tests/learner.flow.test.ts (8 tests) – End-to-end learner practice cycle
✓ tests/problems.audit.test.ts (6 tests) – All 4 seeded problems evaluated end-to-end
```

```
Test Files 5 passed (5)
Tests 61 passed (61)
```

Frontend Test Suite (9 tests):

```
cd frontend && npm test
```

```
✓ src/__tests__/frontend.test.tsx (9 tests) – Problem library, error states, attempt history, score display, feedback cards
```

TypeScript & Production Build Verification:

```
cd backend && npx tsc --noEmit
cd frontend && npm run build
```

Both commands complete with 0 errors and 0 warnings.

Known Limitations & Intentional Trade-Offs

- **No Authentication:** The platform is intentionally structured as an open, focused single-user learner workbench. Excluding user authentication, session cookies, and login flows was a deliberate product decision to focus engineering effort on domain modeling, deterministic evaluation, and iterative feedback quality.
- **Text-First Submissions:** Solutions are submitted as structured Markdown text. While text captures class responsibilities, interfaces, patterns, and trade-offs, interactive drag-and-drop UML drawing is left as future work (see Change Test A).
- **PostgreSQL Dependency:** Relational integrity and atomic transactions rely on PostgreSQL.
- **Synchronous Evaluation Orchestration:** Evaluations are executed synchronously on the backend HTTP thread with concurrency-safe locking. For extreme scale, an asynchronous queue (e.g. BullMQ / Redis) can be adopted without database schema migrations.
- **No Human Reviewer in Prototype:** While the system currently routes to `LLMEvaluator`, the `Evaluator` interface is completely decoupled to support `HumanEvaluator` or `RuleBasedEvaluator` (see Change Test B).

Documentation Links

- [Research Note](#) — Learner problem, market analysis, observed facts vs. assumptions.
- [Design Note](#) — System architecture, domain invariants, and state machine.
- [AI Usage Note](#) — 5 critical engineering decisions and trade-offs.

- [API Specification](#) — Complete REST endpoint schemas and payload examples.
- [LLM Evaluator Architecture](#) — Prompt engineering, evidence grounding, and provider abstraction.

AI Usage & Engineering Decisions

In accordance with the assignment brief, this document outlines 5 meaningful engineering decisions where AI assistance was utilized, critically evaluated, and either accepted, rejected, or modified using senior engineering judgment.

Decision 1: Monolithic Architecture vs. Microservices

- **What AI Suggested:** Early architecture suggestions proposed splitting the system into separate microservices: an `AttemptService`, an `EvaluationService`, an asynchronous event broker, and a dedicated database per service.
 - **What Was Accepted / Rejected: Rejected.** The assignment explicitly evaluates Low-Level Design (domain modeling, class relationships, and object responsibilities), not complex distributed systems.
 - **Why:** Introducing microservices would have added distributed transaction overhead, network latency, serialization boundaries, and deployment friction without any tangible pedagogical benefit for a prototype with 4 problems.
 - **What We Implemented Instead:** A clean, modular monolith with distinct domain, service, and controller layers, unified under an Express/TypeScript backend backed by PostgreSQL.
-

Decision 2: Arbitrary "Score out of 100" vs. Structured Rubric Scoring

- **What AI Suggested:** An initial prompt suggested asking the LLM: "Grade this design out of 100 and summarize why."
 - **What Was Accepted / Rejected: Rejected.**
 - **Why:** Unconstrained LLM scoring produces non-reproducible, arbitrary numbers (e.g., 78 one run, 86 the next) without clear justification. It also hallucinates total scores that do not correlate with specific architectural dimensions.
 - **What Was Modified & Implemented:** We designed a formal, database-backed 7-dimension evaluation rubric totaling 100 points. The LLM is constrained to output structured per-criterion scores, direct quotes of evidence from the student's submission, specific architectural concerns, and actionable suggestions. The backend—not the LLM—deterministically calculates the total score by summing criterion scores.
-

Decision 3: Submission 1:N Evaluation vs. Strict 1:1 Cardinality with In-Place Retries

- **What AI Suggested:** Allowing `Submission 1 : N Evaluation` so that whenever an evaluation fails or is retried, the system simply creates a new `Evaluation` record.
 - **What Was Accepted / Rejected: Rejected** after architectural review.
 - **Why:** In a practice workflow, one specific submitted design has exactly one definitive evaluation. Creating multiple evaluation rows creates ambiguity over which evaluation is current, clutters historical views, and risks multiple evaluation records being created concurrently for the same submission.
 - **What Was Modified & Implemented:** Enforced strict `1 : 1` cardinality with a database-level unique constraint (`UNIQUE(submission_id)`). If an evaluation fails (e.g. LLM timeout), the existing record transitions to `FAILED`. A retry updates the same record in-place (`FAILED → EVALUATING → COMPLETED`). If the student wants to modify their design, they start a new `Attempt`, creating a new `Submission` and `Evaluation`.
-

Decision 4: Evaluator Interface Abstraction (Change Test B)

- **What AI Suggested:** Hardcoding the OpenAI or Gemini SDK directly into the Express route handler or `EvaluationService`.
- **What Was Accepted / Rejected: Rejected the direct coupling; accepted the Evaluator interface pattern.**
- **Why:** Tying the domain practice loop directly to a third-party LLM provider violates the Dependency Inversion Principle and fails **Change Test B** (adding rule-based or human evaluators in the future).
- **What Was Modified & Implemented:** Created an `Evaluator` interface:

```
export interface Evaluator {  
  readonly name: string;
```

```
evaluate(input: EvaluationInput): Promise<EvaluationResult>;
}
```

`LLMEvaluator` implements this contract. The service layer interacts solely with the abstraction. We added a deterministic `MockLLMProvider` so the evaluation pipeline can be tested offline without external API calls.

Decision 5: Direct LLM JSON Trust vs. Strict Zod Schema & Evidence Grounding

- **What AI Suggested:** Relying on basic `JSON.parse(llmOutput)` and persisting the payload directly, trusting the LLM to compute the `totalScore`, match criterion IDs, and validate that its cited evidence is accurate.
- **What Was Accepted / Rejected: Rejected direct trust; implemented multi-stage runtime validation.**
- **Why:** LLMs frequently suffer from arithmetic hallucination (e.g., criterion sub-scores sum to 81, but the LLM states `totalScore: 89`), criterion omission or duplication, and evidence fabrication (quoting concepts or patterns the candidate never mentioned, or providing vague generic feedback like "Looks good").
- **What Was Modified & Implemented:** We built a dedicated `OutputValidator` powered by Zod and an evidence-grounding engine:
 1. **Schema Validation:** Validates the presence of all 7 rubric criteria, enforces numerical ranges (`$0 \le \text{score} \le \text{maxScore}`), and rejects unknown or duplicate criterion IDs.
 2. **Evidence Grounding Verification:** Enforces that each criterion's cited evidence either directly quotes phrases/tokens present in the candidate's submission text or explicitly documents an omission (using markers such as `"not specified"`, `"missing"`, or `"does not mention"`). Rejects generic placeholders and fabricated quotes.
 3. **Deterministic Aggregation:** Ignores any LLM-reported `totalScore` and deterministically computes `totalScore = sum(criterionScores)` on the backend.